

PROJECT REPORT

Smart Grocery Billing System

A Python console application for itemised billing with item-specific coupons, cart coupons, phone-based rewards and tax

Student Name	Arush Pratap Singh
Registration Number	26BCE10968
Course	CSE Fall Sem. 2026
Institute / University	VIT Bhopal Uni.
Guide / Instructor	Abha Trivedi
Language and Tools	Python 3, unittest, Git / GitHub
GitHub Repository	https://github.com/arush26bce10968-sudo/mini-grocery-system
Date of Submission	29 September 2026

Table of Contents

Table of Contents	2
1. Introduction	3
2. Problem Statement	4
3. Functional Requirements	5
4. Non-functional Requirements	5
5. System Architecture	6
6. Design Diagrams	7
7. Design Decisions and Rationale	12
8. Implementation Details	13
9. Screenshots and Results	14
10. Testing Approach	17
11. Challenges Faced	18
12. Learnings and Key Takeaways	19
13. Future Enhancements	19
14. References	20

1. Introduction

Billing at a small grocery store is a repetitive task: the cashier looks up the price of every item, multiplies it by the quantity, applies any discount coupon the customer holds, adds tax and produces a total. Doing this by hand is slow and error-prone, especially when different coupons apply to different items.

The **Smart Grocery Billing System** is a menu-driven Python console application that automates this workflow. The user enters the items and quantities purchased, may add or remove items, supplies item-specific coupon codes, receives a reward coupon after entering a valid phone number, applies a cart-level coupon, and finally receives a neatly formatted bill with a 5% tax.

The project started as a single script that mixed input, calculation and output. During development it was found to contain several logic errors (documented in Section 11). It was therefore restructured into small, single-purpose functions with input validation and a suite of automated unit tests.

Objectives

- Automate price calculation for a fixed catalogue of eight grocery items.
- Support three item-specific coupons and three cart-level coupons.
- Validate all user input so that wrong input never crashes the program.
- Separate business logic from the user interface so the logic can be tested independently.
- Produce an accurate, readable bill including discount, tax and grand total.

2. Problem Statement

Small shops need a quick and reliable way to calculate a customer's bill when the price list is fixed but the purchase, discounts and tax vary from customer to customer. Manual calculation causes arithmetic mistakes, inconsistent discount application and slow checkout.

Problem: Design and implement a console-based billing program that (a) accepts a list of purchased items and quantities, (b) allows the purchase to be edited before billing, (c) applies item-specific and cart-level discount coupons using clearly defined rules, (d) rewards customers who share a valid phone number with a coupon code, and (e) prints an itemised bill with tax and a grand total.

Scope

- In scope: console interaction, in-memory price list, coupon logic, tax calculation, input validation, unit tests.
- Out of scope (see Future Enhancements): graphical interface, persistent storage of bills, payment processing, inventory tracking.

Price list and coupon rules

Item	Price (Rs.)	Item	Price (Rs.)
bread	20	chips	10
butter	50	candy	5
cheese	80	toothpast	40
chicken	300	kurkure	20

Coupon code	Type	Effect
RISEUP	Item-specific	Butter costs 50% less (Rs. 25 per unit)
CLOCKADOODLEDO	Item-specific	Toothpaste costs one-third less (2/3 of price)
SMOOTHLIKE	Item-specific	Cheese costs one-third less (2/3 of price)
MONEY	Cart	30% off the subtotal
MONEYHEIST	Cart	20% off the subtotal (also the phone reward code)
FATHER	Cart	40% off the subtotal

3. Functional Requirements

ID	Requirement
FR1	The system shall display the price list of all eight items at start-up.
FR2	The system shall accept a space-separated list of item names and a matching list of integer quantities.
FR3	The system shall reject unknown items, non-numeric or non-positive quantities, and lists of different lengths, and ask the user to re-enter.
FR4	The system shall merge repeated entries of the same item into one line with the summed quantity.
FR5	The user shall be able to add one item with a quantity to the cart.
FR6	The user shall be able to remove an item from the cart; the system shall report if the item is not present.
FR7	The system shall accept item-specific coupon codes (RISEUP, CLOCKADOODLEDO, SMOOTHLIKE) and reduce the unit price of the related item.
FR8	The system shall issue the reward coupon MONEYHEIST when a valid 10-digit phone number is entered.
FR9	The system shall accept cart-level coupon codes (MONEY, MONEYHEIST, FATHER) case-insensitively and apply the single best (largest) discount.
FR10	The system shall ignore invalid coupon codes without failing.
FR11	The system shall add 5% tax on the discounted total.
FR12	The system shall print an itemised bill: item, quantity, unit price, line amount, subtotal, discount, total before tax, tax and grand total.

4. Non-functional Requirements

Attribute	Requirement
Correctness	All monetary results are rounded to two decimals and verified by automated tests.
Robustness	Invalid input must never terminate the program with an exception; the user gets a clear message instead.
Usability	Prompts are clearly bannered; item and coupon entry is case-insensitive; the bill is column-aligned and readable.
Maintainability	Prices, coupons and tax rate are stored in constants at the top of the file; logic is in small functions with a single responsibility.
Testability	Core logic performs no input or printing, so it can be unit-tested directly.
Performance	Bill generation is $O(n)$ in the number of cart items; response is instantaneous for any realistic cart.
Portability	Uses only the Python 3 standard library (no third-party packages); runs on Windows, Linux and macOS.
Privacy	The phone number is only length-checked; it is never stored or logged.

5. System Architecture

The application follows a simple **three-layer architecture** inside a single Python module. Each layer only talks to the layer beneath it, which keeps the code easy to understand and test.

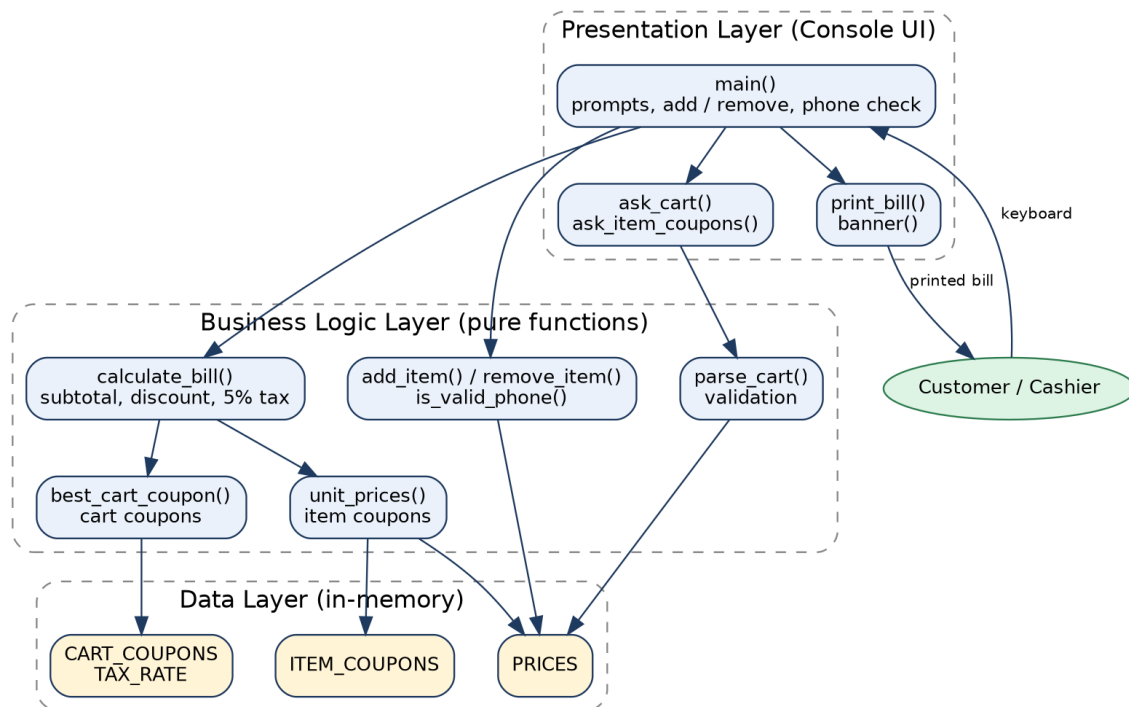


Figure 1: Layered architecture of the billing system

- **Presentation layer** - `main()`, `ask_cart()`, `ask_item_coupons()` and `print_bill()`. Handles every `input()` and `print()`. Contains no pricing rules.
- **Business logic layer** - `parse_cart()`, `unit_prices()`, `best_cart_coupon()`, `calculate_bill()`, `add_item()`, `remove_item()`, `is_valid_phone()`. Pure functions with no console I/O.
- **Data layer** - dictionaries `PRICES`, `ITEM_COUPONS`, `CART_COUPONS` and the constant `TAX_RATE`. Held in memory; changing a price or adding a coupon means editing one line.

6. Design Diagrams

6.1 Use Case Diagram

A single actor, the Customer / Cashier, interacts with the system. All eight use cases are initiated by this actor.

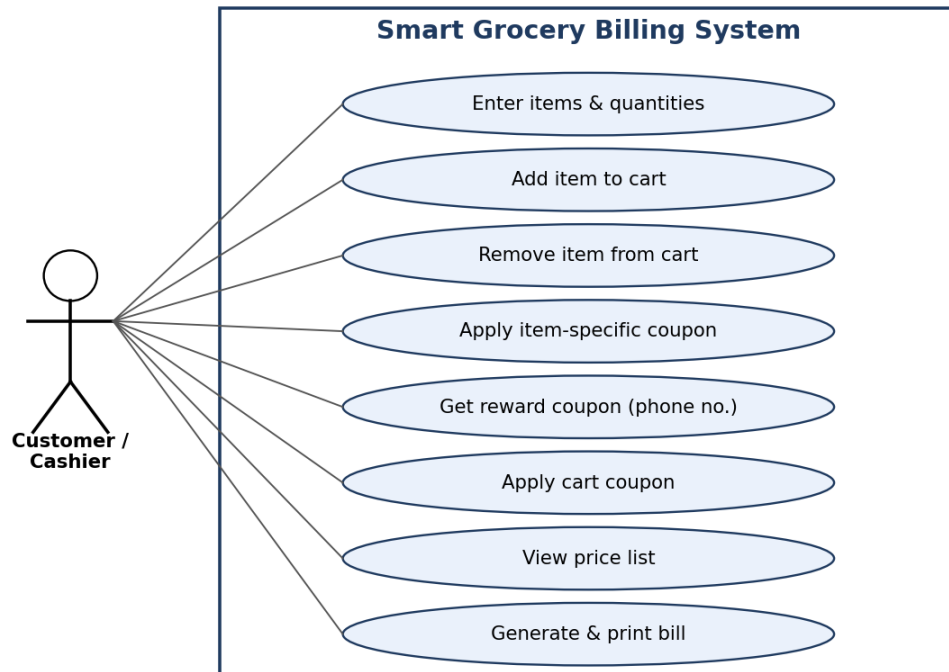


Figure 2: Use case diagram

6.2 Workflow Diagram

The flowchart shows the complete program flow, including the validation loop at the beginning and the early exit when the cart becomes empty after removals.

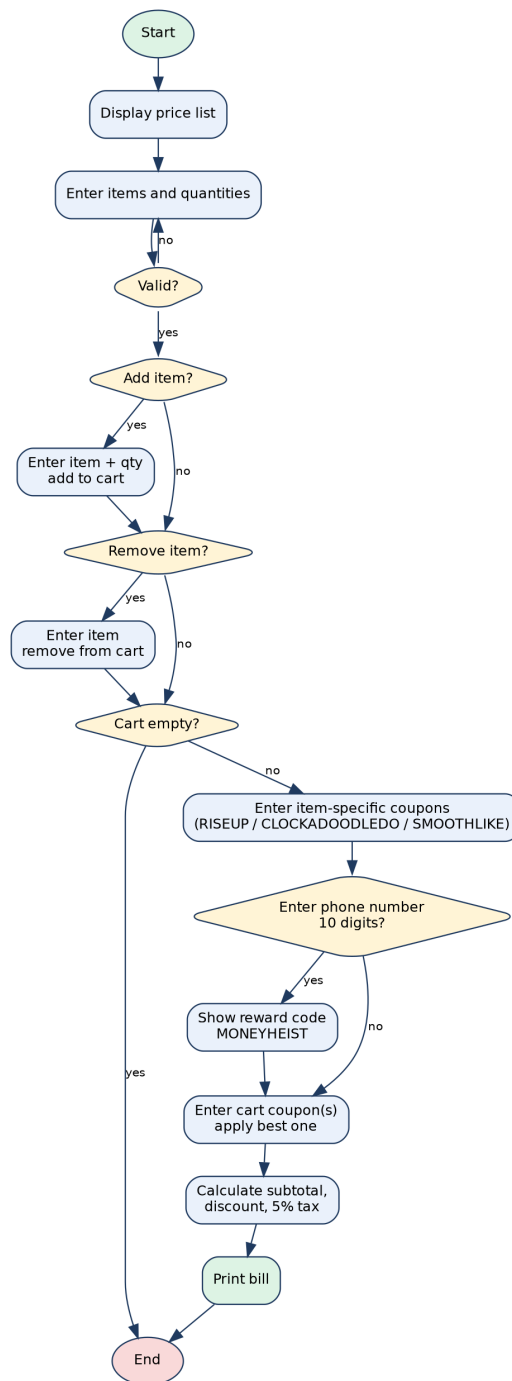


Figure 3: Workflow (flowchart) of the billing process

6.3 Sequence Diagram

The sequence diagram shows the message flow for a typical successful run between the customer, the console UI, the business logic and the data constants.

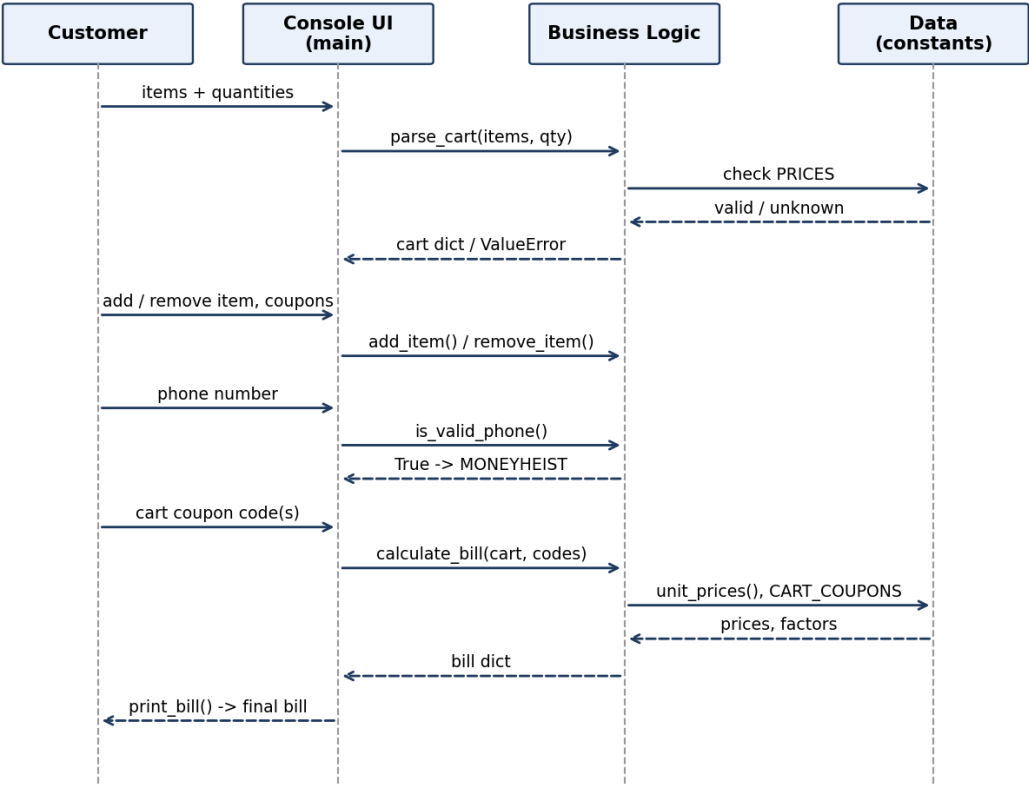


Figure 4: Sequence diagram of a billing session

6.4 Class / Component Diagram

The program uses a functional style with module-level constants rather than user-defined classes. The diagram therefore shows the four components (UI, business logic, data, and the returned Bill structure) together with the test suite that verifies the logic.

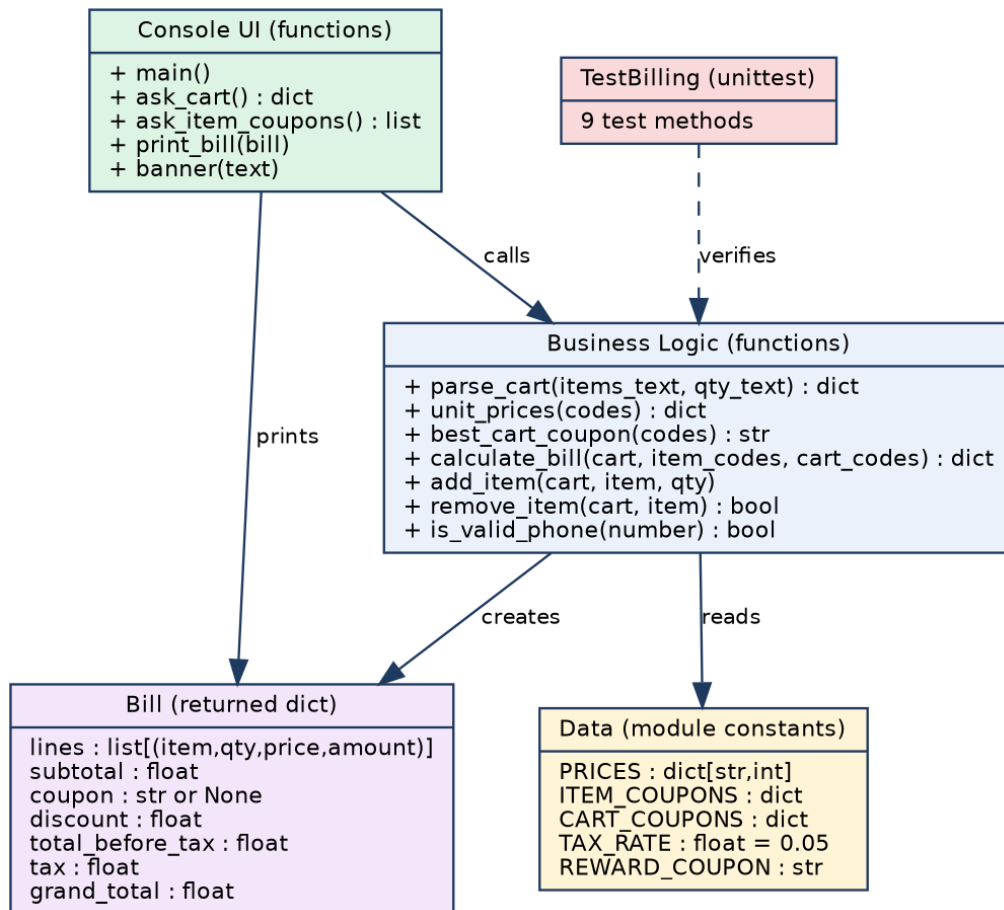


Figure 5: Component diagram

6.5 ER Diagram

Not applicable to the current version: the application keeps all data in memory and stores nothing permanently. The diagram below is the *proposed* schema for the persistent-storage enhancement described in Section 13, where the price list, coupons and every generated bill would be saved in a database.

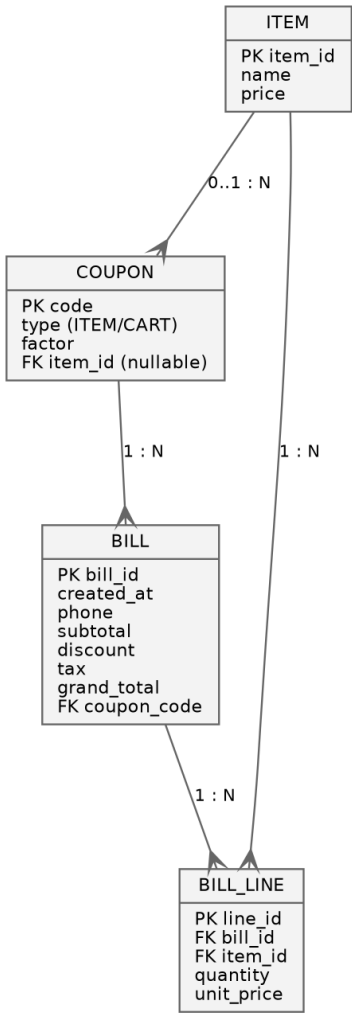


Figure 6: Proposed ER diagram for a future database version

7. Design Decisions and Rationale

Decision	Rationale
Use dictionaries for prices and coupons instead of separate lists / arrays	The original code kept names, prices and coupon effects in unrelated lists and used hard-coded index numbers, which caused wrong prices and <code>IndexError</code> . A dictionary maps an item name directly to its price, so no index can ever be wrong.
Remove the NumPy dependency	NumPy was only used to hold small lists. Plain Python lists and dictionaries are simpler, need no installation and avoid <code>from numpy import *</code> , which silently overrides built-ins such as <code>sum</code> , <code>min</code> , <code>max</code> and <code>round</code> .
Separate logic from input / output	Functions that return values can be unit-tested without typing input. It also lets the interface be replaced (for example by a GUI) without touching the pricing rules.
Validate input in one place (<code>parse_cart</code>) and raise <code>ValueError</code>	One function checks item names, quantities and list lengths; the UI catches the error, shows the message and asks again.
Apply only the best cart coupon	The original design multiplied every coupon entered in sequence, so entering <code>FATHER</code> and <code>MONEY</code> together gave 58% off. Applying only the largest single discount is the usual retail rule and prevents unintended stacking.
Item coupons change the unit price before totals are computed	The discount then appears directly in the price column of the bill, which is transparent to the customer, and cart coupons apply to the already reduced subtotal.
Tax is calculated after discount	Tax is charged on the amount the customer actually pays.
Merge duplicate items	Entering 'bread bread' with quantities '1 2' produces a single line of quantity 3.
Keep the spelling 'toothpast'	This is the name used in the original price list; it is kept so existing usage does not break. It can be corrected in one place (the <code>PRICES</code> dictionary).
Case-insensitive input	'Bread', 'BREAD' and 'bread' behave the same, matching how people actually type.

8. Implementation Details

The whole program is in one file, `billing_system.py`, with tests in `test_billing.py`. The main functions are described below.

Function	Responsibility
<code>parse_cart(items_text, qty_text)</code>	Splits both input lines, converts quantities to integers, validates everything and returns a {item: quantity} dictionary. Raises <code>ValueError</code> with a friendly message on any problem.
<code>unit_prices(codes)</code>	Copies <code>PRICES</code> and, for each valid item coupon, multiplies the related item's price by the coupon factor.
<code>best_cart_coupon(codes)</code>	Filters the entered codes to the valid ones and returns the one with the smallest multiplier (largest discount), or <code>None</code> .
<code>calculate_bill(cart, item_codes, cart_codes)</code>	Builds bill lines, subtotal, applies the cart coupon, computes 5% tax and returns everything in a dictionary rounded to two decimals.
<code>add_item / remove_item</code>	Modify the cart dictionary; <code>add_item</code> validates the name and quantity, <code>remove_item</code> returns <code>True</code> or <code>False</code> .
<code>is_valid_phone(number)</code>	<code>True</code> only for exactly 10 digits.
<code>ask_cart / ask_item_coupons</code>	Loop until valid input is received; invalid coupon codes are ignored with a message.
<code>print_bill(bill)</code>	Prints the column-aligned bill with the coupon line shown only when a coupon was used.
<code>main()</code>	Runs the complete interaction in the order shown in the workflow diagram.

Billing algorithm

1. `prices` = `PRICES` with item-specific coupons applied
2. `subtotal` = `sum(prices[item] * quantity for each item in cart)`
3. `coupon` = best valid cart coupon entered (or none)
4. `after` = `subtotal * coupon_factor` (factor = 1 if no coupon)
5. `tax` = `after * 0.05`
6. `grand` = `after + tax`

Key code excerpt

```
def calculate_bill(cart, item_coupon_codes=(), cart_coupon_codes=()):
    prices = unit_prices(item_coupon_codes)
    lines = [(item, qty, prices[item], prices[item] * qty)
              for item, qty in cart.items()]
    subtotal = sum(line[3] for line in lines)

    coupon = best_cart_coupon(cart_coupon_codes)
    factor = CART_COUPONS[coupon] if coupon else 1.0
    after_discount = subtotal * factor
    tax = after_discount * TAX_RATE
    return {"lines": lines, "subtotal": round(subtotal, 2), ...}
```

Worked example (Run 1 in Section 9): bread 2 x 20 = 40; butter 1 x 25 (RISEUP) = 25; chicken 1 x 300 = 300; cheese 3 x 80 = 240. Subtotal = 605. `FATHER` and `MONEY` were both entered, so the larger discount `FATHER` (40%) applies: 605 x 0.60 = 363.00. Tax = 363 x 0.05 = 18.15. Grand total = **381.15**.

9. Screenshots and Results

The screenshots below are the actual output of the program. Three scenarios were executed: a full run using every feature, a run showing input validation, and a plain run without coupons.

Run 1 - full flow

```
python billing_system.py - Run 1 (full flow): price list, cart, add / remove

>>>>>>>> ITEM PRICES <<<<<<<<<<
1. bread = 20 RS.
2. butter = 50 RS.
3. cheese = 80 RS.
4. chicken = 300 RS.
5. chips = 10 RS.
6. candy = 5 RS.
7. toothpast = 40 RS.
8. kurkure = 20 RS.
>>>>>>>> LIST OF ITEM PURCHASED (separate by space) <<<<<<<<<<
bread butter chicken candy
>>>>>>>> ENTER QUANTITY OF ITEM PURCHASED (same order) <<<<<<<<<<
2 1 1 10

>>>>>>>> DO YOU WANT TO ADD ITEM? <<<<<<<<<<
yes
ENTER ITEM TO ADD: cheese
ENTER QUANTITY: 3

>>>>>>>> DO YOU WANT TO REMOVE ITEM? <<<<<<<<<<
yes
ENTER ITEM TO REMOVE: candy
ITEM REMOVED SUCCESSFULLY

>>>>>>>> ENTER NUMBER OF ITEM SPECIFIC COUPON CODES <<<<<<<<<<
1
>>>>>>>> ENTER ITEM SPECIFIC COUPON CODE <<<<<<<<<<
riseup
```

Figure 7: Price list, cart entry, add item (cheese x 3), remove item (candy), item coupon RISEUP

```
python billing_system.py - Run 1 (continued): coupons and final bill

>>>>>>>> TO GET COUPON ENTER YOUR PHONE NUMBER <<<<<<<<<<
9876543210
>>>>>>>> YOUR COUPON CODE <<<<<<<<<<
v
MONEYHEIST

>>>>>>>> ENTER CART COUPON CODE(S) IF ANY (separate by space) <<<<<<<<<<
father money

ITEM      QTY    PRICE  AMOUNT
-----
bread      2     20.00   40.00
butter     1     25.00   25.00
chicken    1    300.00  300.00
cheese     3     80.00  240.00
-----
Sub total                605.00
Coupon (FATHER)          -242.00
The Total price before tax = 363.00
Tax (5%)                  = 18.15
The Grand Total          = 381.15
----- Thank you for purchase -----
```

Figure 8: Reward coupon, cart coupons FATHER + MONEY (best one applied) and the final bill

Run 2 - error handling

An unknown item (egg) and a quantity list of the wrong length are both rejected with a clear message and the user is asked again. An invalid phone number gives no reward coupon.

```
python billing_system.py - Run 2: input validation and invalid phone number

>>>>>>>> LIST OF ITEM PURCHASED (separate by space) <<<<<<<<<
bread egg
>>>>>>>> ENTER QUANTITY OF ITEM PURCHASED (same order) <<<<<<<<<
1 2
! Unknown item(s): egg Please try again.

>>>>>>>> LIST OF ITEM PURCHASED (separate by space) <<<<<<<<<
bread butter
>>>>>>>> ENTER QUANTITY OF ITEM PURCHASED (same order) <<<<<<<<<
1
! You entered 2 item(s) but 1 quantity value(s). Please try again.

>>>>>>>> LIST OF ITEM PURCHASED (separate by space) <<<<<<<<<
bread butter
>>>>>>>> ENTER QUANTITY OF ITEM PURCHASED (same order) <<<<<<<<<
1 2

>>>>>>>> DO YOU WANT TO ADD ITEM? <<<<<<<<<
no

>>>>>>>> DO YOU WANT TO REMOVE ITEM? <<<<<<<<<
no

>>>>>>>> ENTER NUMBER OF ITEM SPECIFIC COUPON CODES <<<<<<<<<
0

>>>>>>>> TO GET COUPON ENTER YOUR PHONE NUMBER <<<<<<<<<
12345
Invalid Number >>> No Coupon For You

>>>>>>>> ENTER CART COUPON CODE(S) IF ANY (separate by space) <<<<<<<<<
xyz

ITEM      QTY    PRICE    AMOUNT
-----
bread      1     20.00    20.00
butter     2     50.00   100.00
-----
Sub total                120.00
The Total price before tax = 120.00
Tax (5%)                  = 6.00
The Grand Total           = 126.00
----- Thank you for purchase -----
```

Figure 9: Input validation and invalid phone number

Run 3 - no coupons

```
python billing_system.py - Run 3: simple purchase without coupons

>>>>>>> LIST OF ITEM PURCHASED (separate by space) <<<<<<<<<
chips kurkure
>>>>>>> ENTER QUANTITY OF ITEM PURCHASED (same order) <<<<<<<<<
5 3

>>>>>>> DO YOU WANT TO ADD ITEM? <<<<<<<<<
no

>>>>>>> DO YOU WANT TO REMOVE ITEM? <<<<<<<<<
no

>>>>>>> ENTER NUMBER OF ITEM SPECIFIC COUPON CODES <<<<<<<<<
0

>>>>>>> TO GET COUPON ENTER YOUR PHONE NUMBER <<<<<<<<<
98765
Invalid Number >>> No Coupon For You

>>>>>>> ENTER CART COUPON CODE(S) IF ANY (separate by space) <<<<<<<<<

ITEM      QTY    PRICE  AMOUNT
-----
chips      5     10.00   50.00
kurkure    3     20.00   60.00
-----
Sub total                110.00
The Total price before tax = 110.00
Tax (5%)                  = 5.50
The Grand Total          = 115.50
----- Thank you for purchase -----
```

Figure 10: 5 chips + 3 kurkure with no coupons: 110.00 + 5% tax = 115.50

Result summary

Scenario	Subtotal	Discount	Before tax	Tax	Grand total
Run 1 (FATHER, RISEUP)	605.00	242.00	363.00	18.15	381.15
Run 2 (no coupon)	120.00	0.00	120.00	6.00	126.00
Run 3 (no coupon)	110.00	0.00	110.00	5.50	115.50

10. Testing Approach

Testing combined **automated unit tests** on the business logic with **manual scenario tests** of the console flow. Because the logic layer has no input or output, tests call the functions directly with known values and compare the result with hand-calculated answers. The tests use Python's built-in unittest module and run with `python -m unittest -v test_billing`.

Unit tests (all 9 pass)

Test	What it verifies	Expected
test_basic_bill	2 bread + 1 butter with no coupons	Subtotal 90, tax 4.50, total 94.50
test_item_coupon_riseup	2 butter with RISEUP	Subtotal 50
test_item_coupons_toothpaste_and_cheese	3 toothpaste and 3 cheese with two coupons	Subtotal 240
test_best_cart_coupon_chosen	money + father + moneyheist entered	father chosen; 300 becomes 180, total 189
test_invalid_coupon_ignored	Unknown cart coupon 'xyz'	Discount 0
test_bad_input_raises	Unknown item, wrong length, zero quantity, text quantity, empty input	ValueError each time
test_duplicate_items_merge	'bread bread' with '1 2'	{'bread': 3}
test_add_remove	Add chips, remove bread twice, add unknown egg	True, False, ValueError
test_phone	9876543210, 12345, 98765abcde	True, False, False

Manual test cases

#	Input	Expected behaviour	Result
1	Items 'bread egg', qty '1 2'	Rejected: unknown item, asks again	Pass
2	Items 'bread butter', qty '1'	Rejected: length mismatch, asks again	Pass
3	Add item 'milk'	Message that item is not sold; nothing added	Pass
4	Remove item not in cart	Prints ITEM NOT FOUND	Pass
5	Remove every item	'Cart is empty' and program ends cleanly	Pass
6	Phone '12345'	Invalid number, no coupon	Pass
7	Cart coupon 'FATHER' in capitals	40% discount (case-insensitive)	Pass
8	Coupon 'xyz'	Ignored, no discount	Pass
9	Two cart coupons 'father money'	Only the larger discount applied	Pass

11. Challenges Faced

The original version of the program ran only on very simple input. The main challenge was to find and repair the hidden logic errors and then to redesign the code so that such errors would be caught early. The table lists the defects that were found.

#	Problem in the original code	Effect	Fix
1	<code>k.lower</code> and <code>m.lower</code> used without brackets	Compared a method object with a string, so every coupon and item check was False; no discount was ever applied and item prices were never added.	Call <code>.lower()</code> ; later replaced by dictionary lookups.
2	<code>l4 = array(), str</code> followed by <code>l4.append(c)</code>	NumPy arrays have no append method; the program crashed with <code>AttributeError</code> when any item coupon count was above 0.	Use a plain Python list.
3	<code>range(0, d+1)</code>	Asked for one more coupon than the user declared (off-by-one).	<code>for _ in range(count)</code>
4	Hard-coded <code>l3 = array([20, 50, 300])</code> and access such as <code>l3[6]</code> , <code>l3[3]</code>	Prices were positional guesses; index 3 and 6 do not exist, causing <code>IndexError</code> , and the coupons changed the wrong items.	Named dictionary of prices; coupons refer to items by name.
5	<code>a = a + l3[i]</code> executed for every item	Added an unrelated price to the bill for each item, inflating the total; also an <code>IndexError</code> for more than three items.	Removed; total is the sum of price x quantity only.
6	Price list contained 'egg' branch but egg is not in the price list; 'chicken' used a non-existent index	Inconsistent catalogue.	One PRICES dictionary is the single source of truth.
7	Variable named <code>list</code> and <code>from numpy import *</code>	Shadowed Python built-ins, a source of confusing bugs.	Meaningful names and no star import.
8	No validation of items, quantities or lengths	Typos or mismatched inputs caused wrong bills or crashes.	<code>parse_cart</code> validates and the UI re-asks.
9	Cart coupons applied one after another	Combinations stacked (for example 58% off).	Only the best single cart coupon applies.
10	Duplicate items counted as separate lines	Cluttered bill.	Quantities of the same item are merged.

Other challenges were deciding the sensible rule for combining coupons, keeping the interface unchanged for users familiar with the old version while improving it, and testing an interactive program. The last was solved by moving all calculations into functions that can be tested without typing input.

12. Learnings and Key Takeaways

- **Data structures matter.** Choosing a dictionary instead of parallel lists removed a whole class of index errors.
- **Small functions are easier to get right.** Each function does one job and can be understood, tested and changed independently.
- **Separate logic from I/O.** This single decision made automated testing possible.
- **Never trust user input.** Every input path needs validation and a friendly error message.
- **Common Python pitfalls:** forgetting brackets on a method call (`.lower` vs `.lower()`), off-by-one loop ranges, and shadowing built-in names.
- **Tests give confidence.** After every change the nine tests confirmed that earlier behaviour still worked.
- **Requirements need clear rules.** Ambiguities such as how coupons combine must be decided and documented, otherwise the program behaves unpredictably.
- **Diagrams help design.** Drawing the workflow and sequence before refactoring exposed missing steps such as the empty-cart case.

13. Future Enhancements

- **Database storage** (SQLite) for prices, coupons and past bills, as in the proposed ER diagram.
- **Graphical interface** using Tkinter or a web front end (Flask / Django) reusing the existing business logic unchanged.
- **Inventory management:** reduce stock on each sale and warn on low stock.
- **Coupon expiry, usage limits** and one-time coupons tied to a customer's phone number.
- **Bill export** as PDF or a printable receipt and sending it by SMS or e-mail.
- **Barcode scanning** to enter items automatically.
- **Multiple payment modes** (cash, UPI, card) and change calculation.
- **Reports** such as daily sales, best-selling items and coupon usage.
- **Configurable tax slabs** per item category (GST) loaded from a settings file.
- **Item catalogue in a file** (JSON / CSV) so the shopkeeper can update prices without editing code.

14. References

- [1] Python Software Foundation. *The Python Tutorial and Python Standard Library*.
<https://docs.python.org/3/>
- [2] Python Software Foundation. *unittest - Unit testing framework*.
<https://docs.python.org/3/library/unittest.html>
- [3] Python Software Foundation. *Built-in Types: dict*.
<https://docs.python.org/3/library/stdtypes.html#mapping-types-dict>
- [4] van Rossum, G., Warsaw, B., Coghlan, N. *PEP 8 - Style Guide for Python Code*.
<https://peps.python.org/pep-0008/>
- [5] Graphviz Project. *DOT Language documentation* (used for the flow, architecture, component and ER diagrams). <https://graphviz.org/doc/info/lang.html>
- [6] Hunter, J. D. *Matplotlib documentation* (used for the use case and sequence diagrams and screenshots). <https://matplotlib.org/>
- [7] ReportLab Inc. *ReportLab PDF Library User Guide* (used to generate this report).
<https://docs.reportlab.com/>
- [8] Fowler, M. *UML Distilled: A Brief Guide to the Standard Object Modeling Language*. Addison-Wesley.
- [9] Sommerville, I. *Software Engineering*. Pearson.